Unit 1 – Using Objects and Methods
Lesson 7: String Manipulation



String Data Types

- There are two kinds of data types in Java: _____ data types and _____ data types (also called reference data types).
    - Class data types are capitalized and primitive data types are not
- Primitive data types only hold _____ of data at a time. `int` and `double` are both primitive data types; they hold one number at a time.
- When we declare a primitive data type, Java allocates space in memory and stores the data there, in the same space as the information that tells us what the variable is called.
- When we declare a class data type, Java allocates space in memory for a memory address, and that memory address points to the location of the data.

	Primitive Data Types	*Class Data Types*
Initializing	`int x = 5;`	`String example = "Hello";` `String lessCommon = new String("Hello");`
Memory		
Assignment	`int x = 5;` `int y = x;`	`String a = "Hello";` `String b = a;`

- For Strings, knowing what is going on in memory doesn't make much difference to us, because the String class is what we call _____
- This means once we set up a String in memory, we can't actually change it – all we can do is make the variable point at a different location in memory
- Other mutable class data types allow us to actually change what's stored in the memory location – if a and b point to the same object, changes to that object will be reflected in both variables

Code Segment	**Console**

`String first = "Harry Potter";`
`String second = alpha;`
`first = first + " and the Goblet of Fire";`
`System.out.println(first);`
`System.out.println(second);`	

- Variables of class data types can be empty.
- We call a class variable which is contains no data: _____
    - `String example;`
    - `String otherExample = null;`

Concatenation

- String addition is also called _____, and all it does is glue two Strings together.
- We can also use the += operator to add-on to a String we have already created. (This is actually creating new String reference, because Strings cannot be edited)

Code Segment	**Console**

`String h = "Hello";`
`h += "World";`
`System.out.println(h);`	

If Java is already dealing with Strings then it will take any primitive value (e.g. an `int` or `double`) it finds and implicitly convert that to a string:

Code Segment	Console

`int x = 5;`
`int y = 3;`
`System.out.println("The sum is: " + x + y);`	

This happens because of the way Java allocates memory. Once it starts working with Strings, it continues to look at things at Strings, and interprets '+' as a concatenation operation. We can circumvent this by taking advantage of the order of operations:

Code Segment	Console

`int x = 5;`
`int y = 3;`
`System.out.println("The sum is: " + (x + y));`	

Escape Sequences

- Escape sequences allow us to print off special symbols or commands in a String literal
- All escape sequences start with a _____ and the next symbol after it tells Java what special case to print off
  - To include a literal quote in a String: _____
  - To include a new line in a String: _____
  - To include a tab in a String: _____
  - To include a literal single quote in a String: _____
  - To include a literal backslash in a String: _____

Code Segment	Console
`System.out.println("Hello\nWorld”);`	
`System.out.println(“\”Hello\tWorld\””);`	

String Indexing

- String literals are made up of letter, digits, and characters
- Each symbol in a String literal is assigned an _____
- In Java, indexes in a String literal start at _____ and _____
- Escape sequences, even though they are technically two symbols, count as a single index

Example:

```
String message = "AP CS is a fun class!";
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
A	P		C	S		i	s		a		f	u	n		c	l	a	s	s	!

Libraries and APIs

- _____ are collections of pre-written classes that you can use in your programs to save time and effort
- An _____ (Application Programming Interface) is a guide that tells you how to use the classes in a library
- API Documentation explains the attributes (fields) and behaviors (methods) of a class, helping you understand how to use it
- Classes in libraries are grouped into packages. For example:
    - The `java.lang` package includes classes like `String` and `Math`
- You can use these classes to create objects and perform tasks without writing everything from scratch

String Methods

- There are 5 methods you must know for the AP CSA exam that deal with Strings
- These 5 methods are part of the `java.lang` package, which is automatically imported with each Java program, so no import statement is needed
- Each of these method calls returns information about the String literal that called it.

```
String message = "AP CS is a fun class!";
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
A	P		C	S		i	s		a		f	u	n		c	l	a	s	s	!

Method Returns	*Method*	*What does it do?*	*Example*	*Result*
`int`	`.length()`	Gives you the length of the String	`message.length()`	
`String`	`.substring(int from, int to)`	Returns the substring beginning at from and ending at (to-1)	`message.substring(6,17)`	
`String`	`.substring(int from)`	Returns substring from to length()	`message.substring(15)`	
`int`	`.indexOf(String str)`	Return the index of the first occurrence of str; returns -1 if not found.	`message.indexOf("a")` `message.indexOf("I")`	

`int`	`.compareTo(String other)`
	
	*Don't worry about the value; concern yourself with if the number is positive, negative, or 0
		• Returns a value < 0 if "this" is lexicographically less than "other"
		• Returns a value = 0 if "this" is lexicographically equal to "other"
		• Returns a value > 0 if "this" is lexicographically greater than "other"
			`String first = "c";`
			`first.compareTo("a")`
			`first.compareTo("g")`
			`first.compareTo("c")`	

*Lexicographically – dictionary order … think of this:

a	b	c	d	e	f	g	h
1	2	3	4	5	6	7	8

Methods Practice

```
String words = "AP CS is a fun class!";
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
A	P		C	S		i	s		a		f	u	n		c	l	a	s	s	!

What is the result of the following method calls?

`words.substring(0, 2)`	
`words.substring(11, 14)`	
`words.substring(15, 21)`	
`words.substring(0, 1)`	
`words.substring(6, 7)`	
`words.substring(0, 0)`	
`words.substring(20, 21)`	
`words.substring(-1, 2)`	
`words.substring(0)`	
`words.substring(11)`	
`words.substring(6, words.length())`	
`words.substring(3, words.length() - 1)`	
`words.substring(3, 14).length()`	
`words.substring(words.indexOf("a")).substring(6)`	

Class Methods vs. Reference Methods

- The methods we use with Strings are called _____, meaning we need a reference to the variable in order to use them:

```
String digits = "43897543";
String value = digits.substring(3);
```

- In the example above, `digits` is our reference variable used to call our substring method
- The methods we will use with our `Math` class are called _____, meaning we use the class name to call the method we use:

```
double result = Math.sqrt(25);
```

- In the example above, we use `Math`, the name of the class, to call on the square root method

Class Methods (Static Methods)	**Reference Methods (Instance Methods)**
• Class methods belong to the class itself, not to specific objects • You call them using the class name • Example: The `Math` class has `static` methods, so you use `Math.methodName()`	• Reference methods belong to an object of a class and operate on that specific object • You call them using an object reference • Example: The `String` class has instance methods, so you use `stringObject.methodName()`